Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la lucrarea de laborator nr. 7

Disciplina: Tehnici de programare

Tema: GitHub și depanare
Arsenalul sub lupă

A efectuat: Moraru Elena, grupa SI-264

A verificat: asistent universitar Reițman Patricia

Chișinău 2026

Lucrarea A. GitHub, pas cu pas

Am inițializat depozitele locale exercitiu-git și exercitiu-2 pe ramura main. În primul am rulat salut.py, am salvat fișierul într-un commit și am adăugat regulile .gitignore pentru .venv și __pycache__. Al doilea conține fișierul README.md și primul commit.

```
1  print("Salut, GitHub!")
```

Fig. 1.1. Fișierul salut.py în editor.

```
1  > git --version
2  git version 2.55.0.windows.5
3
4  > python salut.py
5  Salut, GitHub!
6
7  > git check-ignore .venv/ __pycache__/
8  .venv/
9  __pycache__/
10
11 > git diff -- salut.py
12 diff --git a/salut.py b/salut.py
13 index e700475..b7c6a14 100644
14 --- a/salut.py
15 +++ b/salut.py
16 @@ -1 +1,2 @@
17  print("Salut, GitHub!")
18 +print("al doilea")
19
20 > git restore -- salut.py
21
22 > git status
23 On branch main
24 nothing to commit, working tree clean
```

Fig. 1.2. Rularea scriptului, regulile de ignorare, diferența și restaurarea fișierului.

Stadiu local. Publicarea celor două depozite și verificarea operațiilor clone, push și pull pe GitHub rămân de realizat.

Lucrarea B. Jurnalul depanării

Am pornit de la scriptul cu erori din fișa B. Am transcris în tinte.json cele șase intrări afișate pe ELSE, inclusiv portul „abc” și intrarea fără câmpul serviciu. Am salvat fiecare reparație într-un commit separat.

1. SyntaxError (ab443ce). Am adăugat două puncte după antetul funcției incarca.

2. IndentationError (e306125). Am indentat return None în blocul except.

3. NameError (22afbfb). Am corectat numele variabilei tinta în tinte.

4. TypeError (8995394). Am afișat numărul țintelor printr-un f-string.

5. KeyError (f57d213). Am folosit valoarea necunoscut când câmpul serviciu lipsește.

6. AttributeError (753598e). Am returnat dicționarul rezultat din pe_servicii.

7. TypeError (5a1b3d4). Am convertit argumentul --prag la întreg prin argparse.

8. Rezultat gol (e8853c0). Am mutat return după bucla de filtrare.

9. ValueError (2a8007c). Am verificat porturile și am sărit peste valorile invalide la filtrare.

10. Port lipsă (d4c8c69). Am păstrat lista fiecărui serviciu când am adăugat un port.

Pentru pragurile 1000 și 1024, rezultatul conține aceleași trei gazde. Verificarea a comparat toate liniile afișate și conținutul fișierului raport_inventar.json cu valorile așteptate.

Citirea, gruparea și filtrarea

```python
def incarca(cale):
    """Citeste lista de tinte din fisierul JSON."""
    with open(cale) as f:
        return json.load(f)


def port_valid(text):
    """Intoarce portul ca int, sau None daca nu e un numar."""
    try:
        return int(text)
    except ValueError:
        return None
```

Fig. 2.1. Citirea fișierului JSON și conversia porturilor.

```python
def pe_servicii(tinte):
    """Dict: serviciu -> lista de porturi."""
    grupe = {}
    for t in tinte:
        serviciu = t.get("serviciu", "necunoscut")
        port = port_valid(t["port"])
        if port is None:
            print("sar peste", t["gazda"], ": port invalid", t["port"])
            continue
        grupe.setdefault(serviciu, []).append(port)
    return grupe


def peste_prag(tinte, prag):
    """Lista gazdelor cu un port mai mare decat pragul."""
    rezultat = []
    for t in tinte:
        port = port_valid(t["port"])
        if port is None:
            continue
        if port > prag:
            rezultat.append(t["gazda"])
    return rezultat
```

Fig. 2.2. Gruparea porturilor pe servicii și filtrarea după prag.

Pentru o țintă fără serviciu, raportul folosește categoria necunoscut. Porturile valide se acumulează în lista fiecărui serviciu, iar filtrarea trece prin toate țintele.

Rularea programului reparat

```python
def main():
    p = argparse.ArgumentParser()
    p.add_argument("fisier")
    p.add_argument("--prag", type=int, default=1024)
    a = p.parse_args()

    tinte = incarca(a.fisier)
    print(f"Tinte citite: {len(tinte)}")

    grupe = pe_servicii(tinte)
    for serviciu, porturi in grupe.items():
        print(f"{serviciu}: {porturi}")

    mari = peste_prag(tinte, a.prag)
    print(f"Peste {a.prag}: {mari}")

    with open("raport_inventar.json", "w") as f:
        json.dump({"servicii": grupe, "peste_prag": mari}, f, indent=2)
```

Fig. 2.3. Argumentele programului și scrierea raportului JSON.

```
> python inventar.py tinte.json --prag 1000
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [80, 8080]
mysql: [3306]
necunoscut: [5432]
Peste 1000: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
Raport salvat in raport_inventar.json

> python inventar.py tinte.json
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [80, 8080]
mysql: [3306]
necunoscut: [5432]
Peste 1024: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
Raport salvat in raport_inventar.json
```

Fig. 2.4. Ieșirea salvată pentru pragul 1000 și pragul implicit 1024.

Grupul HTTP conține porturile 80 și 8080. Portul abc este ignorat.

Anexă. Codul sursă

salut.py

```
print("Salut, GitHub!")
```

inventar.py

```
# inventar.py - rezumatul tintelor dintr-un fisier JSON
# Unealta a fost scrisa in graba si nu mai merge. Reparati-o.
# Rulare:  python inventar.py tinte.json --prag 1000
import argparse
import json


def incarca(cale):
    """Citeste lista de tinte din fisierul JSON."""
    with open(cale) as f:
        return json.load(f)


def port_valid(text):
    """Intoarce portul ca int, sau None daca nu e un numar."""
    try:
        return int(text)
    except ValueError:
        return None
```

Anexă. inventar.py (continuare)

```
def pe_servicii(tinte):
    """Dict: serviciu -> lista de porturi."""
    grupe = {}
    for t in tinte:
        serviciu = t.get("serviciu", "necunoscut")
        port = port_valid(t["port"])
        if port is None:
            print("sar peste", t["gazda"], ": port invalid", t["port"])
            continue
        grupe.setdefault(serviciu, []).append(port)
    return grupe


def peste_prag(tinte, prag):
    """Lista gazdelor cu un port mai mare decat pragul."""
    rezultat = []
    for t in tinte:
        port = port_valid(t["port"])
        if port is None:
            continue
        if port > prag:
            rezultat.append(t["gazda"])
    return rezultat


def main():
    p = argparse.ArgumentParser()
    p.add_argument("fisier")
    p.add_argument("--prag", type=int, default=1024)
    a = p.parse_args()

    tinte = incarca(a.fisier)
    print(f"Tinte citite: {len(tinte)}")

    grupe = pe_servicii(tinte)
    for serviciu, porturi in grupe.items():
        print(f"{serviciu}: {porturi}")

    mari = peste_prag(tinte, a.prag)
    print(f"Peste {a.prag}: {mari}")

    with open("raport_inventar.json", "w") as f:
        json.dump({"servicii": grupe, "peste_prag": mari}, f, indent=2)
    print("Raport salvat in raport_inventar.json")


if __name__ == "__main__":
    main()
```